BrainAI

ProSuite Industrial

INDUSTRIAL ENCRYPTION SUITE

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**ProEDC System Integrations-Handbuch**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2025	S. Köhne	Stable	Initial Release of Wrapper Suite
2.0.0	27.02.2026	S. Köhne	Stable	Full API Integration (Hashing, KeyGen, Crypto)

1 Produktübersicht

Die **ProEDC Wrapper Suite** ist das Bindeglied zwischen hochsprachenbasierten Applikationen und der nativen ProEDC-Plattform. Sie ermöglicht den Zugriff auf das gesamte Funktionsspektrum der Suite: von der hardwarenahen Entropiegewinnung über Hochgeschwindigkeits-Hashing bis hin zur hybriden Dateiverschlüsselung.

Hauptkomponenten der Wrapper-API:

- **ProEDC High-Level API:** Einfache Methoden für Datei-Operationen und Initialisierung.
- **ProKey Integration:** Direkter Zugriff auf die native Entropie-Quelle zur Schlüsselerzeugung.
- **ProHash Integration:** Low-Level Zugriff auf den Hashing-Kontext für Datenströme und Dateien.
- **ProED Core:** Granulare Kontrolle über Verschlüsselungskontexte und IV-Handling.

2 Gemeinsame Datenstrukturen & Status

Um die binäre Kompatibilität zu gewährleisten, bilden alle Wrapper die nativen C-Strukturen exakt ab.

2.1 ProEDC_Status (Fehlerbehandlung)

Konstante	Wert	Beschreibung
`SUCCESS`	0	Operation erfolgreich.
`ERR_PARAM`	-1	Ungültige Parameter (z.B. falsche Schlüssellänge).
`ERR_LIMIT_REACHED`	-7	**Lizenz-Stopp:** Das Volumenlimit der Edition wurde erreicht.

2.2 ProHash_Ctx (Hashing-Kontext)

Wird verwendet, um Hashes über große Datenmengen in Chunks zu berechnen. Die Struktur umfasst den internen Zustand (`belt`), den Byte-Zähler (`count`) und den Puffer (`buffer`).

3 C# (.NET) – Tiefenintegration

Der C#-Wrapper nutzt `unsafe` Pointer-Arithmetik für maximale Performance und bietet direkten Zugriff auf alle Core-Module.

3.1 Dateiverschlüsselung & Hashing

```
using ProEDC;

// 1. Suite initialisieren (Lizenz & Treiber-Check)
ProNative.ProEDC_Init();

// 2. Einen neuen 256-Bit Key generieren (via ProKey)
byte[] myKey = new byte[32];
fixed (byte* pKey = myKey) {
    ProNative.ProKey_Generate(pKey, (UIntPtr)32);
}

// 3. Datei verschlüsseln (mit integriertem Lizenz-Check)
ProEDC_Status st = ProNative.ProEDC_EncryptFile("source.dat", "enc.da
if (st == ProEDC_Status.ERR_LIMIT_REACHED) HandleUpgrade();

// 4. ProHash Kontext für Stream-Hashing nutzen
ProHash_Ctx hCtx = ProNative.CreateHashContext();
ProNative.prohash_init(&hCtx);
// ... prohash_update & prohash_final ...
```

4 Java (JNA) – Enterprise Standard

Die Java-Implementierung nutzt JNA-Strukturen mit striktem Speicher-Alignment (4-Byte für Context, 8-Byte für Hash).

4.1 Automatisierte Datei-Operationen

```
import com.brainai.proedc.ProNative;

// High-Level Client nutzt die proedc.dll / .so
// Erstellt Schlüssel, berechnet Hashes und verarbeitet Dateien
byte[] key = ProNative.Client.generateKey(32); // Nutzt ProKey
String fileHash = ProNative.Client.calculateHash(someData); // Nutzt

int status = ProNative.Client.encryptFile("input.db", "output.pro", "
```

5 Python – Automatisierung & Analyse

Der Python-Wrapper bildet die `ctypes`-Strukturen für `ProEDContext` und `ProHashCtx` ab, um native Performance in Skripten zu erreichen.

5.1 Vollständiges Funktionsbeispiel

```
from proedc import ProEDC

client = ProEDC()

# 1. Datei-Integrität prüfen
hash_val = client.calculate_hash(b"System-Integrity-Check") # ProHash

# 2. Key-Management
new_key = client.generate_key(32) # ProKey

# 3. Commercial File API
status = client.encrypt_file("config.json", "config.enc", "master.key
if status == 0:
    print(f"Verschlüsselung abgeschlossen. Hash: {hash_val}")
```

6 JavaScript (Node.js) – Backend Security

Nutzt `ffi-napi` und `ref-struct`, um die 72-Byte großen nativen Kontexte direkt im Node.js-Speicher zu verwalten.

6.1 Integration

```
const proedc = require('./proedc');

// Initialisiert die Engine (ProEDC_Init)
const key = proedc.generateKey(32); // Hardware-Entropie
const hash = proedc.calculateHash("Data Stream Content"); // ProHash-

// Status-Codes werden direkt als Integer zurückgegeben
const res = proedc.encryptFile('archive.tar', 'archive.proedc', 'key.
```

7 VB.NET – OT & Legacy Support

Die VB-Implementierung nutzt `MarshalAs`-Attribute, um die fixen Array-Größen der nativen Strukturen (`SizeConst:=8` für State, `SizeConst:=64` für Puffer) sicherzustellen.

7.1 Beispiel

```
Imports ProEDC

' Suite und Version prüfen
ProNative.ProEDC_Init()
Dim ver As String = ProNative.GetVersionString()

' Dateizugriff via Commercial API
Dim st As ProEDCStatus = ProNative.ProEDC_EncryptFile("data.bin", "da
```

8 Best Practices für Wrapper-Entwickler

- **Initialisierung:** Rufen Sie `ProEDC_Init()` einmalig beim Anwendungsstart auf. Dies stellt sicher, dass Lizenzprüfungen und Hardware-Bypass-Checks (PRO Edition) korrekt ausgeführt werden.
- **Lizenz-Handling:** Implementieren Sie immer eine Prüfung auf `ERR_LIMIT_REACHED` (-7). Dies ist keine technische Störung, sondern ein administratives Ereignis (TRIAL/INDUSTRIAL Volumen erschöpft).
- **Schlüsselschutz:** Nutzen Sie die `ProKey_Generate`-Methoden der Wrapper, anstatt eigene Zufallsfunktionen zu verwenden, um die volle Entropie der ProSuite zu garantieren.

Hinweis: Diese Dokumentation beschreibt die vollständige API-Schnittstelle. Für details zur internen Implementierung der kryptografischen Kerne konsultieren Sie bitte die internen Spezifikationsdokumente.

© 2026 Sascha Köhne	✉️ **Contact: koehne83@googlemail.com**	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**